DNN Numerical Cheat Sheet: Formulas & How to Solve

---

1. Perceptron Learning Rule

Formulas: - Z = w0 + w1 * x1 + w2 * x2 - Y_pred = sign(Z) - Weight update: Δw = η * (Y_true - Y_pred) * x - New weight: w_new = w_old + Δw

How to Solve: - For each row in truth table: - Compute Z - Apply sign(Z) - Compare to target Y; if wrong, update weights

---

2. Gradient Descent (Standard)

Formula: - $w = w - \eta * \nabla E(w)$

How to Solve: - Compute gradient $\nabla E(w)$ - Update weights using learning rate η

Example: Minimize $E(w) = w^2$ - $\nabla E(w) = 2w$ - $\eta = 0.1$, $w_0 = 2$ - $w_1 = 2 - 0.1 * 2 * 2 = 1.6$ - $w_2 = 1.6 - 0.1 * 2 * 1.6 = 1.28$ - $w_3 = 1.28 - 0.1 * 2 * 1.28 = 1.024$

---

3. Backpropagation

Formulas: - z = W * a + b - a = activation(z) - $\delta = (a - y) \cdot \text{activation}'(z)$ - $\Delta W = \eta * \delta * a_{prev}^T$

How to Solve: - Perform forward pass - Calculate loss - Backpropagate error layer-by-layer - Compute gradients and update weights

---

4. Binary Cross-Entropy Loss Derivative

Formulas: - $L = -[y * \log(a) + (1 - y) * \log(1 - a)]$ - $\partial L / \partial z = a - y$ (where a = sigmoid(z))

How to Solve: - Use predicted output a and true label y - Apply derivative directly in weight updates

---

5. Categorical Cross-Entropy + Softmax

Formulas: - Softmax: $a_i = e^{z_i} / \Sigma e^{z_j}$ - Loss: $L = -\Sigma y_i * \log(a_i)$ - $\partial L / \partial z_k = a_k - y_k$

How to Solve: - Calculate softmax outputs - Use difference (predicted - actual) for gradient

---

6. Computational Graph Derivatives

Concept: - Use chain rule across graph: ∂f/∂x = ∂f/∂u * ∂u/∂x

How to Solve: - Break expression into intermediate nodes - Compute forward values - Backpropagate gradients using chain rule

---

7. Dropout Combinatorics

Formula: - Total networks = 2^n (where n = number of nodes using dropout)

How to Solve: - Count number of units under dropout - Raise 2 to that power

---

8. Parameter Count in a Neural Network

Formulas: - Weights per layer = input_dim * output_dim - Biases = output_dim - Total = Σ (weights + biases) across all layers

How to Solve: - Multiply neurons between connected layers - Sum up across layers including bias

---

9. MLP Architecture for Boolean Functions

Formulas: - For full single hidden layer: 2^n perceptrons - Deep MLP: layers = 2 * ceil(log2(n))

How to Solve: - Count input variables n - Use MLP sizing formulas to estimate minimum structure

---

10. Constructing an MLP for Complex Decision Boundaries

Steps: - Identify linear boundaries dividing classes - Write equations of lines (Ax + By = C form) - For each boundary, define one perceptron with weights = [A, B], bias = -C - First hidden layer implements these boundaries - Output layer combines boundary activations (AND/OR logic)

How to Solve: - Use sign or step activation in hidden nodes - Combine outputs logically in the final layer to get correct classification for all regions

---

11. How to Determine Number of Layers and Hidden Units

For XOR or Complex Boolean Logic: - Number of perceptrons (single layer) = 2^(n - 1) - Number of layers (deep MLP) = $2 \times \text{ceil}(\log_2(n))$

General Tip: - Minimum one hidden layer required to solve non-linear boundaries (e.g., XOR) - Use rule of thumb: Hidden nodes ≈ (input + output) / 2 or try incrementally based on accuracy

---

Use this cheat sheet during revision and solving numericals. Let me know if you'd like a printable PDF version or diagram-based notes.